

AWS Website Deployment on Amazon EC2

Deploying a Secure Static Website Using Ubuntu Linux, Nginx & Route 53

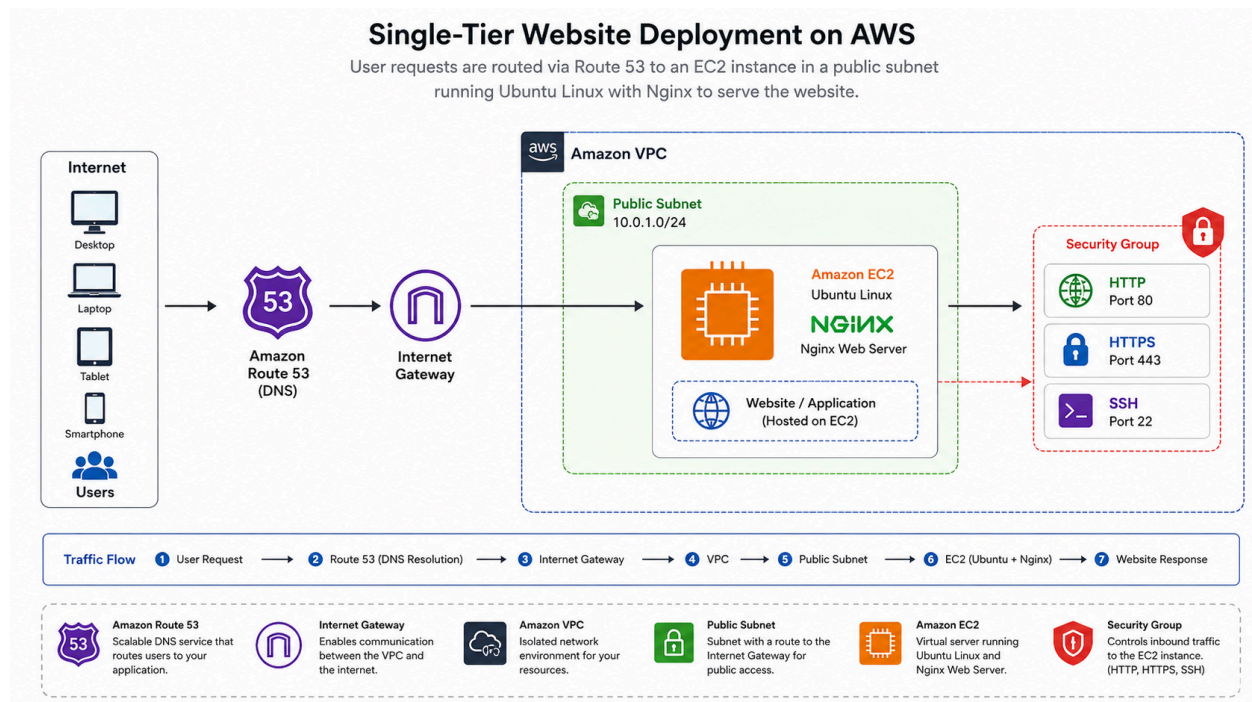
Prepared by:

Nwafor Chukwunonso

Project Overview

This project demonstrates the successful deployment of a secure static website on Amazon Web Services (AWS). The website was hosted on an Amazon EC2 instance running Ubuntu Linux with Nginx configured as the web server. Amazon Route 53 was used for DNS configuration, while AWS Security Groups were implemented to secure network access.

The project showcases practical experience in cloud infrastructure deployment, Linux server administration, web server configuration, DNS management, and AWS networking.



Technologies Used

Cloud Platform

- Amazon Web Services (AWS)

AWS Services

- Amazon EC2
- Amazon VPC
- Amazon Route 53
- AWS Security Groups

Operating System

- Ubuntu Linux

Web Server

- Nginx

Tools

- SSH
- Linux Terminal

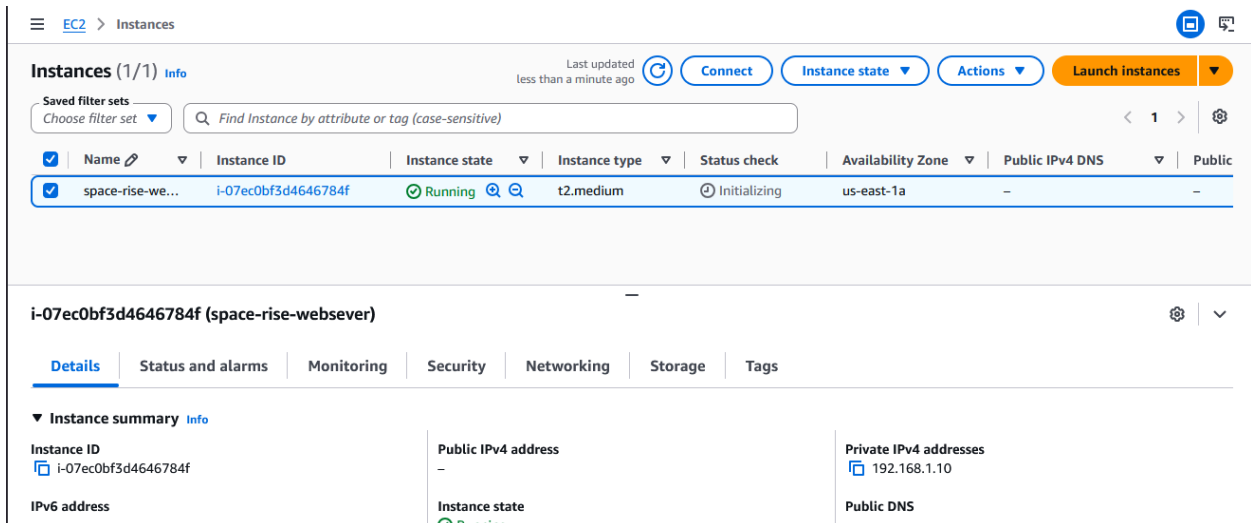
Project Objectives

- Deploy a static website on Amazon EC2.
- Configure Ubuntu Linux for web hosting.
- Install and configure Nginx as the web server.
- Configure DNS using Amazon Route 53.
- Secure the server using AWS Security Groups.
- Verify that the website is publicly accessible.

Implementation Process

Step 1 – Launch an Amazon EC2 Instance

Created and configured an Ubuntu EC2 instance to host the website.



The screenshot shows the Amazon EC2 console 'Instances' page. At the top, there's a navigation bar with 'EC2' and 'Instances'. Below it, a summary bar shows 'Instances (1/1)' and 'Info'. A search bar is present with the text 'Find Instance by attribute or tag (case-sensitive)'. A table lists the instances, with one instance 'space-rise-we...' having ID 'i-07ec0bf3d4646784f', state 'Running', type 't2.medium', and availability zone 'us-east-1a'. Below the table, the details for the selected instance are shown, including tabs for 'Details', 'Status and alarms', 'Monitoring', 'Security', 'Networking', 'Storage', and 'Tags'. The 'Details' tab is active, showing the instance summary with fields for Instance ID, Public IPv4 address, Private IPv4 addresses, IPv6 address, Instance state, and Public DNS.

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public
space-rise-we...	i-07ec0bf3d4646784f	Running	t2.medium	Initializing	us-east-1a	-	-

i-07ec0bf3d4646784f (space-rise-webserver)

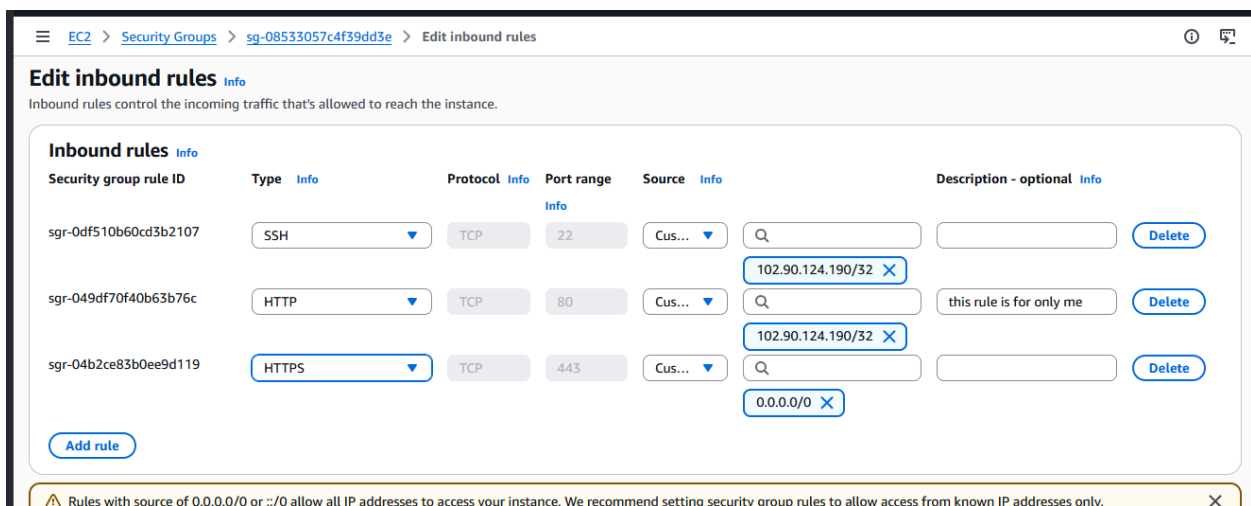
Instance summary

Instance ID	Public IPv4 address	Private IPv4 addresses
i-07ec0bf3d4646784f	-	192.168.1.10
IPv6 address	Instance state	Public DNS
	Running	

Step 2 – Configure Security Groups

Configured inbound rules to allow:

- SSH (Port 22)
- HTTP (Port 80)
- HTTPS (Port 443)

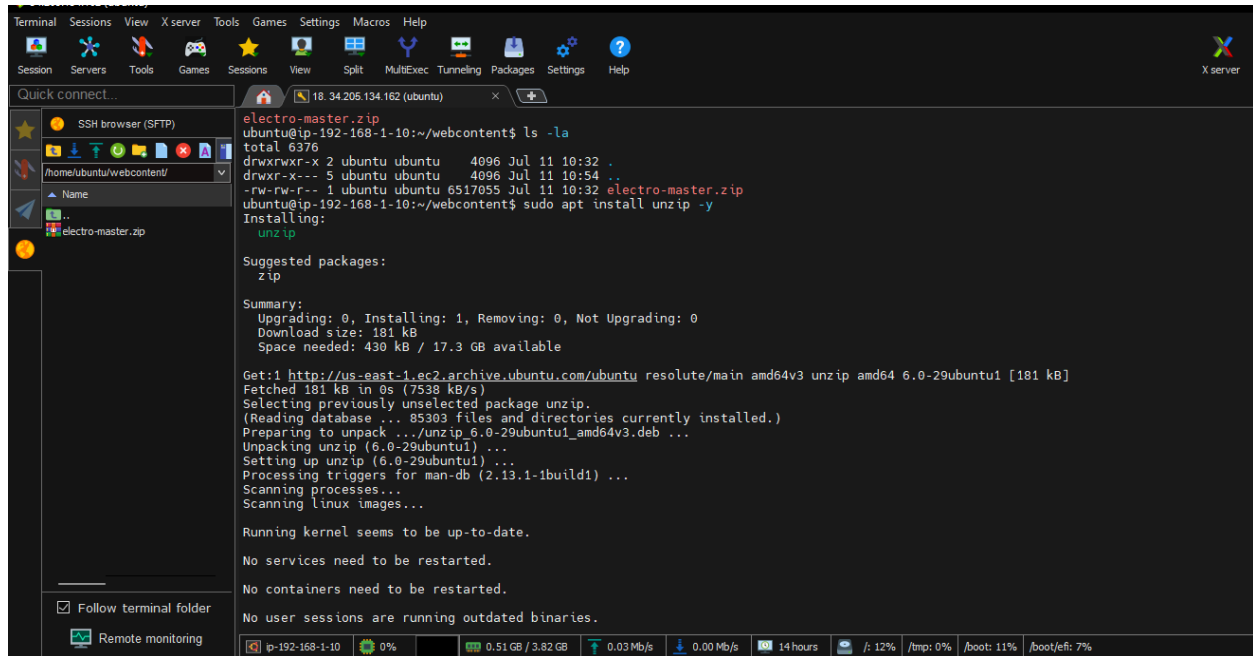


The screenshot shows the 'Edit inbound rules' page for security group 'sg-08533057c4f39dd3e'. The page title is 'Edit inbound rules' with a sub-header 'Inbound rules control the incoming traffic that's allowed to reach the instance.' Below this, there's a table of inbound rules. The table has columns for 'Security group rule ID', 'Type', 'Protocol', 'Port range', 'Source', and 'Description - optional'. There are three rules listed: 'sgr-0df510b60cd3b2107' for SSH (Port 22), 'sgr-049df70f40b63b76c' for HTTP (Port 80), and 'sgr-04b2ce83b0ee9d119' for HTTPS (Port 443). The source for all rules is '0.0.0.0/0'. There are 'Delete' buttons for each rule and an 'Add rule' button at the bottom. A warning message at the bottom states: 'Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.'

Security group rule ID	Type	Protocol	Port range	Source	Description - optional
sgr-0df510b60cd3b2107	SSH	TCP	22	0.0.0.0/0	
sgr-049df70f40b63b76c	HTTP	TCP	80	0.0.0.0/0	this rule is for only me
sgr-04b2ce83b0ee9d119	HTTPS	TCP	443	0.0.0.0/0	

Step 3 – Connect to the Server

Connected securely to the EC2 instance using SSH for remote administration.



Step 4 – Configure Ubuntu Linux

Updated the server packages and prepared the Linux environment for deployment.

Commands used:

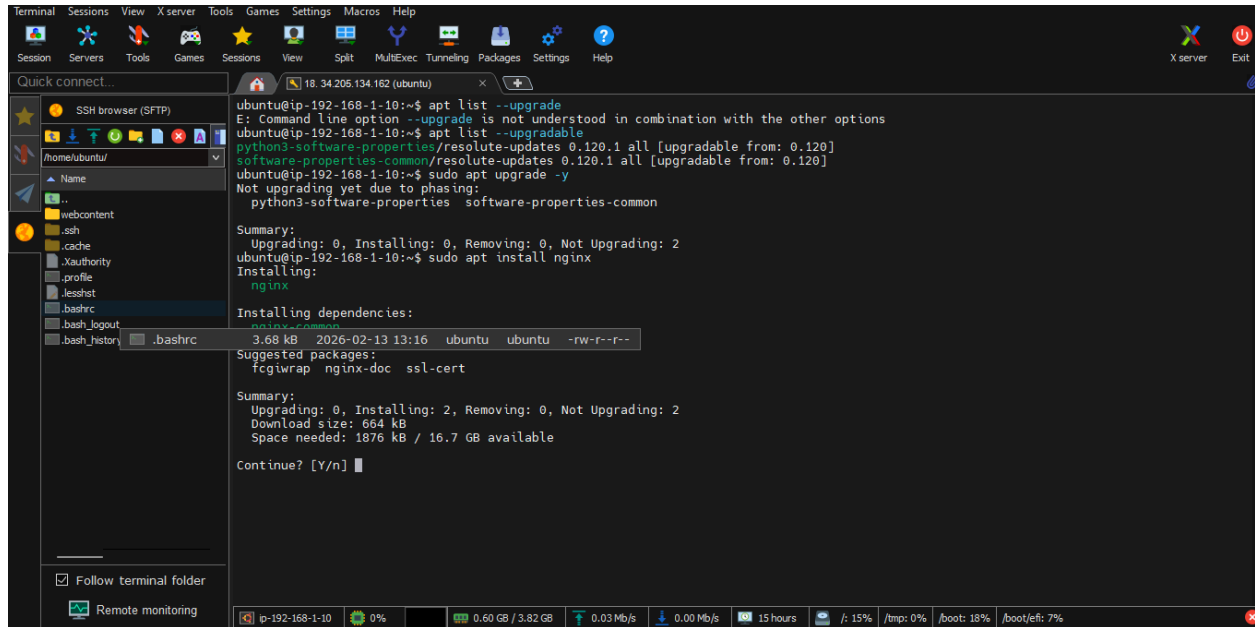
```
sudo apt update  
sudo apt upgrade
```

Step 5 – Install Nginx

Installed and configured Nginx to serve the website.

Commands used:

```
sudo apt install nginx  
sudo systemctl start nginx  
sudo systemctl enable nginx
```



```
ubuntu@ip-192-168-1-10:~$ apt list --upgrade
E: Command line option --upgrade is not understood in combination with the other options
ubuntu@ip-192-168-1-10:~$ apt list --upgradable
python3-software-properties/resolute-updates 0.120.1 all [upgradable from: 0.120]
software-properties-common/resolute-updates 0.120.1 all [upgradable from: 0.120]
ubuntu@ip-192-168-1-10:~$ sudo apt upgrade -y
Not upgrading yet due to phasing:
python3-software-properties software-properties-common

Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2
ubuntu@ip-192-168-1-10:~$ sudo apt install nginx
Installing:
nginx

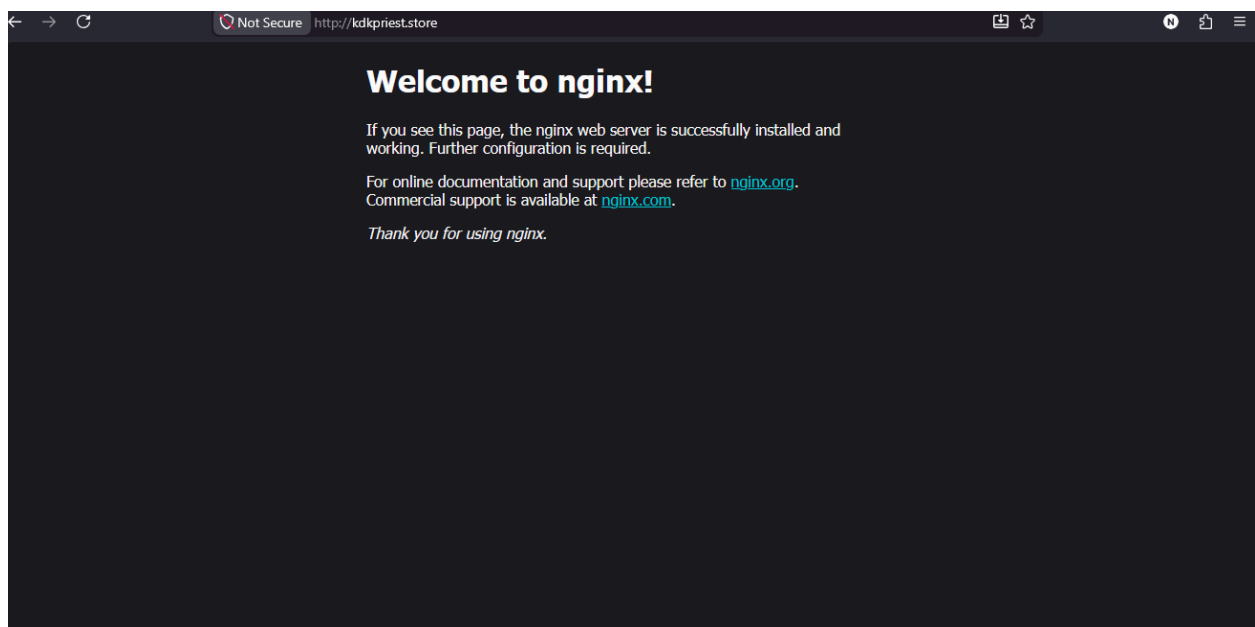
Installing dependencies:
nginx-common
3.68 kB 2026-02-13 13:16 ubuntu ubuntu -rw-r--r--
Suggested packages:
fcgiwrap nginx-doc ssl-cert

Summary:
Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 2
Download size: 664 kB
Space needed: 1876 kB / 16.7 GB available

Continue? [Y/n]
```

Step 6 – Deploy the Website

Uploaded the website files to the Nginx web root directory and verified successful deployment.



Step 7 – Configure Route 53

Configured DNS records to connect the domain name to the EC2 instance.

☰ VPC > Route tables > rtb-063f3e87e0f37f3d8 > Edit routes 🔍

Edit routes

Destination	Target	Status	Propagated	Route Origin
192.168.1.0/28	local	✓ Active	No	CreateRouteTable
Q 0.0.0.0/0	Q local			
	Internet Gateway	✓ Active	No	CreateRoute
	Q igw-05205b098184941db			

Project Outcome

The static website was successfully deployed on Amazon EC2 using Ubuntu Linux and Nginx. DNS resolution was configured using Amazon Route 53, and the infrastructure was secured using AWS Security Groups. The deployment demonstrates foundational cloud engineering skills and best practices for hosting static websites on AWS.

Skills Demonstrated

- AWS Cloud Computing
- Amazon EC2
- Amazon VPC
- Amazon Route 53
- Ubuntu Linux
- Linux Server Administration
- Nginx
- SSH
- DNS Configuration
- Website Deployment
- Cloud Networking
- Cloud Security

Deliverables

- ✓ Amazon EC2 Instance Deployment
- ✓ Ubuntu Linux Server Configuration
- ✓ Nginx Installation and Configuration
- ✓ Static Website Deployment
- ✓ Route 53 DNS Configuration
- ✓ AWS Security Group Configuration
- ✓ SSH Remote Administration
- ✓ Functional Website Verification

Why Choose My Service?

I deploy secure, reliable, and well configured static websites on AWS using industry best practices. Every deployment is carefully configured with security, performance, and scalability in mind. Whether you need a personal portfolio, business website, or landing page, I can help you host it efficiently on Amazon Web Services.

Contact

Nwafor Chukwunonso

AWS Cloud Engineer | DevOps | Cloud Security | Linux

"Building Secure, Scalable & Reliable Cloud Infrastructure on AWS."